



EAST WEST UNIVERSITY

Department of Computer Science and Engineering FALL 2024

Course Code : CSE246
Course Title : Algorithms
Section – 01

Project Report on

Travel Planner Using Graph

Submitted By :

Name : Fayroz Tasnim Rowza
Id : 2022-3-60-011

Department of CSE, EWU

Submitted To :

Name : Dr. Taskeed Jabid
Designation : Professor
Department of CSE
East West University

Date of Report Submission: February 02, 2025

Table of Contents

Page Number

Introduction	03
Problem Statement	03
Technical implementation	03
Source code	03
Output and Snapshots	07
Conclusion	09

Introduction: The Travel Planner is a sophisticated route optimization system that leverages graph theory and advanced algorithms to help users find the most efficient travel routes between locations. By representing the transportation network as a graph data structure, where locations serve as nodes and transportation routes as weighted edges, the system can compute optimal paths based on various criteria such as distance, time, or cost.

Problem Statement: Traditional travel planning often involves manual comparison of multiple routes and transportation options, which can be time-consuming and may not always yield the most efficient results. This project addresses this challenge by automating the route optimization process using well-established graph algorithms.

Technical Implementation: The system is implemented in C and utilizes three fundamental shortest path algorithms:

- **Dijkstra's Algorithm:** Efficiently computes the shortest path between a single source and destination, ideal for specific route queries.
- **Floyd-Warshall Algorithm:** Calculates shortest paths between all pairs of locations, useful for comprehensive route analysis.
- **Bellman-Ford Algorithm:** Handles single-source shortest paths and can detect negative weight cycles, making it suitable for complex transportation networks.

Source code:

```
#include <stdio.h>
#include <limits.h>
#define MAX 100

int graph[MAX][MAX];
int n;

void dijkstra(int source, int destination) {
    int dist[MAX], visited[MAX], parent[MAX];
    for (int i = 0; i < n; i++) {
        dist[i] = INT_MAX;
        visited[i] = 0;
        parent[i] = -1;
    }
    dist[source] = 0;
    for (int count = 0; count < n - 1; count++) {
        int min = INT_MAX, u;
```

```

for (int v = 0; v < n; v++) {
    if (!visited[v] && dist[v] <= min) {
        min = dist[v], u = v;
    }
}
visited[u] = 1;
for (int v = 0; v < n; v++) {
    if (!visited[v] && graph[u][v] && dist[u] != INT_MAX && dist[u] + graph[u][v] < dist[v]) {
        dist[v] = dist[u] + graph[u][v];
        parent[v] = u;
    }
}
printf("\nShortest distance from location %d to location %d: %d\n", source, destination, dist[destination]);
}

```

```

void floydWarshall() {
    int dist[MAX][MAX];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dist[i][j] = graph[i][j];
            if (i != j && graph[i][j] == 0) dist[i][j] = INT_MAX;
        }
    }
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] != INT_MAX && dist[k][j] != INT_MAX && dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
    printf("\nAll-pairs shortest paths computed using Floyd-Warshall Algorithm.\n");
}

```

```

void bellmanFord(int source) {
    int dist[MAX];
    for (int i = 0; i < n; i++) dist[i] = INT_MAX;
}

```

```

dist[source] = 0;
for (int i = 0; i < n - 1; i++) {
    for (int u = 0; u < n; u++) {
        for (int v = 0; v < n; v++) {
            if (graph[u][v] && dist[u] != INT_MAX && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
}
printf("\nShortest paths from location %d using Bellman-Ford Algorithm computed.\n", source);
}

```

```

int main() {
    int choice, source, destination, weightType;
    printf("Enter number of locations: ");
    scanf("%d", &n);
    printf("\nChoose weight type for travel:\n");
    printf("1. Distance\n");
    printf("2. Time\n");
    printf("3. Cost\n");
    printf("Enter your choice: ");
    scanf("%d", &weightType);

    printf("\nEnter weights for every route between locations:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i != j) {
                printf("Enter weight from location %d to location %d: ", i, j);
                scanf("%d", &graph[i][j]);
            } else {
                graph[i][j] = 0;
            }
        }
    }
}

```

```

while (1) {
    printf("\nChoose an algorithm to compute the shortest path:\n");
    printf("1. Dijkstra's Algorithm (Single Source Shortest Path)\n");
}

```

```

printf("2. Floyd-Warshall Algorithm (All Pairs Shortest Paths)\n");
printf("3. Bellman-Ford Algorithm (Single Source Shortest Path)\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
if (choice == 4) break;
if (choice == 1 || choice == 3) {
    printf("\nEnter source location (index 0 to %d): ", n - 1);
    scanf("%d", &source);
    printf("Enter destination location (index 0 to %d): ", n - 1);
    scanf("%d", &destination);
    if (choice == 1) dijkstra(source, destination);
    else bellmanFord(source);
} else if (choice == 2) {
    floydWarshall();
} else {
    printf("Invalid choice, please try again.\n");
}
}
return 0;
}

```

Overview of Source Code:

The code starts with necessary header inclusions and defines a maximum size constant and uses two global variables.

The program contains three main algorithm functions:

- `dijkstra()`: Implements Dijkstra's algorithm to find the shortest path between two points
- `floydWarshall()`: Implements Floyd-Warshall algorithm to find all-pairs shortest paths
- `bellmanFord(source)`: Implements Bellman-Ford algorithm for single-source shortest paths

The “main()” function handles:

- Getting number of locations from user
- Choosing weight type (distance/time/cost)
- Input of weights between locations
- Menu for selecting which algorithm to use
- Getting source and destination locations
- Calling appropriate algorithm function based on user choice

The program uses a simple command-line interface where users can repeatedly choose different algorithms until they select to exit. The graph is stored as an adjacency matrix where each cell represents the weight (distance/time/cost) between two locations.

Output and Snapshots:

Conclusion: The Travel Planner project successfully combines graph theory with practical travel planning. Using three different algorithms (Dijkstra's, Floyd-Warshall, and Bellman-Ford), the system helps users find the best routes between locations based on their preferences for distance, time, or cost. The program provides a straightforward way to input locations and calculate optimal paths. While the current version uses a simple command-line interface, it effectively demonstrates how computer science concepts can solve everyday travel problems. The project not only serves as a useful tool for route planning but also shows how mathematical algorithms can make our daily travel decisions easier and more efficient.